DATA 607 - New York Times API Analysis

Most Popular Articles by News Section

Mohd Afwan Shaikh

2026-11-01

Contents

Project Approach . . . 1
Introduction . . . 2
Setup . . . 2
API Authentication and Data Collection . . . 2
Parsing the JSON Response . . . 3
Creating a Tidy Data Frame . . . 3
Data Cleaning and Validation . . . 4
    Data-Cleaning Decisions . . . 5
Analysis: Most Popular News Sections . . . 5
    Top Five Sections . . . 6
Data Visualization . . . 6
Findings . . . 7
Conclusion . . . 8
References . . . 8

Project Approach

For this assignment, I plan to use the **New York Times Most Popular API** to explore which news sections appear most frequently among the most-viewed articles over the past 30 days. My main research question is: **Which New York Times news sections are most represented among popular articles?** I want to identify the top five sections and compare how frequently they appear in the results.

I will use **R** to connect to the API using a securely stored API key and retrieve the data in JSON format. I will then convert the JSON response into a tidy data frame containing article titles, sections, publication dates, and authors. I plan to use packages such as `httr2`, `dplyr`, and `ggplot2` to collect, clean, analyze, and visualize the data.

Some challenges I expect include missing information, nested JSON fields, API request limits, and differences in how news sections are labeled. I will handle missing values, check for duplicate articles, and make sure the data is organized correctly before analyzing it. Finally, I plan to create a simple bar chart comparing the most frequently represented sections and document my findings in an R Markdown report.

Introduction

The New York Times provides several APIs for accessing article information. For this assignment, I selected the **Most Popular API**, specifically the Most Viewed endpoint, which returns metadata about articles that received high readership during a selected period.

My objective is to identify which news sections are most frequently represented among the returned articles and compare their distribution.

API Endpoint:

`https://api.nytimes.com/svc/mostpopular/v2/viewed/30.json`

The `30` in the endpoint specifies the 30-day period. The request also uses an `api-key` query parameter for authentication.

Setup

The following packages are used to connect to the API, process JSON data, organize the results, and create visualizations.

```
library(httr2)
library(dplyr)
library(purrr)
library(tibble)
library(ggplot2)
library(knitr)
```

API Authentication and Data Collection

To protect my API key, I stored it in the `.Renviron` file and accessed it using `Sys.getenv()`. This avoids including sensitive credentials directly in the report.

```
# Retrieve API key from environment
api_key <- Sys.getenv("NYT_API_KEY")

if (!nzchar(api_key)) {
  stop("NYT_API_KEY is missing. Configure it in .Renviron.")
}

# Define the API endpoint
endpoint <- "https://api.nytimes.com/svc/mostpopular/v2/viewed/30.json"

# Send request to NYT API
response <- request(endpoint) |>
  req_url_query(`api-key` = api_key) |>
  req_perform()

# Confirm successful request
status_code <- resp_status(response)

cat("API Response Status:", status_code)
```

```
## API Response Status: 200
```

Parsing the JSON Response

The API returns data in JSON format. I used `resp_body_json()` to parse the response into an R object.

```
# Convert JSON response to R object
nyt_data <- resp_body_json(
  response,
  simplifyVector = FALSE
)

# Check the API response
cat("API Status:", nyt_data$status, "\n")
```

```
## API Status: OK
```

```
cat("Articles Returned:", length(nyt_data$results))
```

```
## Articles Returned: 20
```

Creating a Tidy Data Frame

The JSON response contains nested information about each article. I extracted six fields: article ID, title, section, publication date, author, and URL.

```
# Extract article fields into a tibble
articles <- map_dfr(nyt_data$results, function(x) {
  tibble(
    article_id = as.character(x$id),
    title = x$title,
    section = x$section,
    published_date = as.Date(x$published_date),
    author = x$byline,
    url = x$url
  )
})

# Display dimensions
cat("Rows:", nrow(articles), "\n")
```

```
## Rows: 20
```

```
cat("Columns:", ncol(articles), "\n")
```

```
## Columns: 6
```

```
# Display first 10 records
kable(
  head(articles, 10),
  caption = "Sample of NYT Most-Viewed Articles"
)
```

Table 1: Sample of NYT Most-Viewed Articles

article_id	title	section	published_date	author	url
100000011143974	Latest Polls	Polls	2026-09-10		https://www.nytimes.com/interactive/polls/latest-polls.html
100000011013475	Patrick Stewart, Master of Shakespeare, 'Star Trek' and More, Has Died	Arts	2026-10-29	By Anita Gates and William McDonald	https://www.nytimes.com/2026/10/29/arts/television/patrick-stewart-dead.html
100000011150452	She Was Offered $40 Million for Silence on Elon Musk. She Spoke Out Instead.	Movies	2026-10-09	By Coralie Kraft	https://www.nytimes.com/2026/10/09/movies/ashley-st-clair-interview-elon-musk.html
100000011238489	'Cheesecake Factory' Jab Rankles Trump on Mississippi Campaign Stop	U.S.	2026-10-26	By Chris Cameron	https://www.nytimes.com/2026/10/26/us/politics/obamas-cheesecake-factory-jab-rankles-trump-on-mississippi-campaign-stop.html
100000011066985	To Keep Cancer in Remission. Why Is No One Talking About It?	Well	2026-10-06	By Nina Agrawal	https://www.nytimes.com/2026/10/06/well/cancer-prevention-exercise-treatment.html
100000011191476	How Cornell Punished Each of the 7 Men Accused of Sexual Assault	New York	2026-10-03	By Sharon Otterman, Hurubie Meko and Debra Kamin	https://www.nytimes.com/2026/10/03/nyregion/cornell-men-university-discipline.html
100000010948076	O.J. Simpson, a Secret Informant and a Decades-Old Mystery	U.S.	2026-10-18	By Mike Baker and Tim Arango	https://www.nytimes.com/2026/10/18/us/oj-simpson-ippolito-informant-accomplice.html
100000011213021	Trump's New Press Secretary: Not-Quite Attorney at Law	U.S.	2026-10-14	By Shawn McCreesh	https://www.nytimes.com/2026/10/14/us/politics/trump-press-secretary-katie-zacharia-lawyer.html
100000011227950	Obama Issues Searing Attacks on Trump and Ramaswamy in Return to Campaign Trail	U.S.	2026-10-24	By Tim Balk, Emily Davies, John Yoon and Richard Fausset	https://www.nytimes.com/2026/10/24/us/politics/obama-trump-midterms-campaign.html
100000011127139	They Argued About Trump at a Dog Park. Then They Pulled Their Guns.	U.S.	2026-10-05	By Jessica Contrera	https://www.nytimes.com/2026/10/05/us/trump-ohio-dog-park-shooting.html

Data Cleaning and Validation

Before analyzing the results, I checked for missing values, blank section names, and duplicate article IDs. I also standardized section names by removing unnecessary whitespace.

```
# Check missing values
missing_values <- colSums(is.na(articles))

kable(
  data.frame(
    Column = names(missing_values),
```

```
    Missing_Values = as.integer(missing_values)
  ),
  caption = "Missing Values by Column"
)
```

Table 2: Missing Values by Column

Column	Missing_Values
article_id	0
title	0
section	0
published_date	0
author	0
url	0

```
# Check duplicates before cleaning
duplicate_count <- sum(duplicated(articles$article_id))

cat("Duplicate Article IDs:", duplicate_count, "\n")
```

```
## Duplicate Article IDs: 0
```

```
# Clean data
articles_clean <- articles |>
  distinct(article_id, .keep_all = TRUE) |>
  mutate(
    section = trimws(section),
    section = if_else(
      is.na(section) | section == "",
      "Unknown",
      section
    ),
    author = na_if(trimws(author), "")
  )

cat("Cleaned Articles:", nrow(articles_clean))
```

```
## Cleaned Articles: 20
```

Data-Cleaning Decisions

I retained the fields most relevant to the research question and excluded nested multimedia information because it was unnecessary for the analysis. I converted publication dates into R's Date format, checked for missing values, replaced missing or blank section names with "Unknown," and removed duplicate article IDs. Blank author names were converted to missing values.

Analysis: Most Popular News Sections

To answer the research question, I grouped the articles by section and counted how frequently each section appeared. I also calculated the percentage of returned articles belonging to each section.

```r
# Count articles by section
section_summary <- articles_clean |>
  count(section, name = "article_count", sort = TRUE) |>
  mutate(
    percentage = round(
      article_count / sum(article_count) * 100,
      1
    )
  )

# Display section summary
kable(
  section_summary,
  caption = "Article Counts and Percentages by Section"
)
```

Table 3: Article Counts and Percentages by Section

section	article_count	percentage
U.S.	11	55
Opinion	3	15
Arts	1	5
Movies	1	5
New York	1	5
Polls	1	5
Technology	1	5
Well	1	5

Top Five Sections

```r
top_five <- section_summary |>
  slice_head(n = 5)

kable(
  top_five,
  caption = "Top Five Most-Represented NYT Sections"
)
```

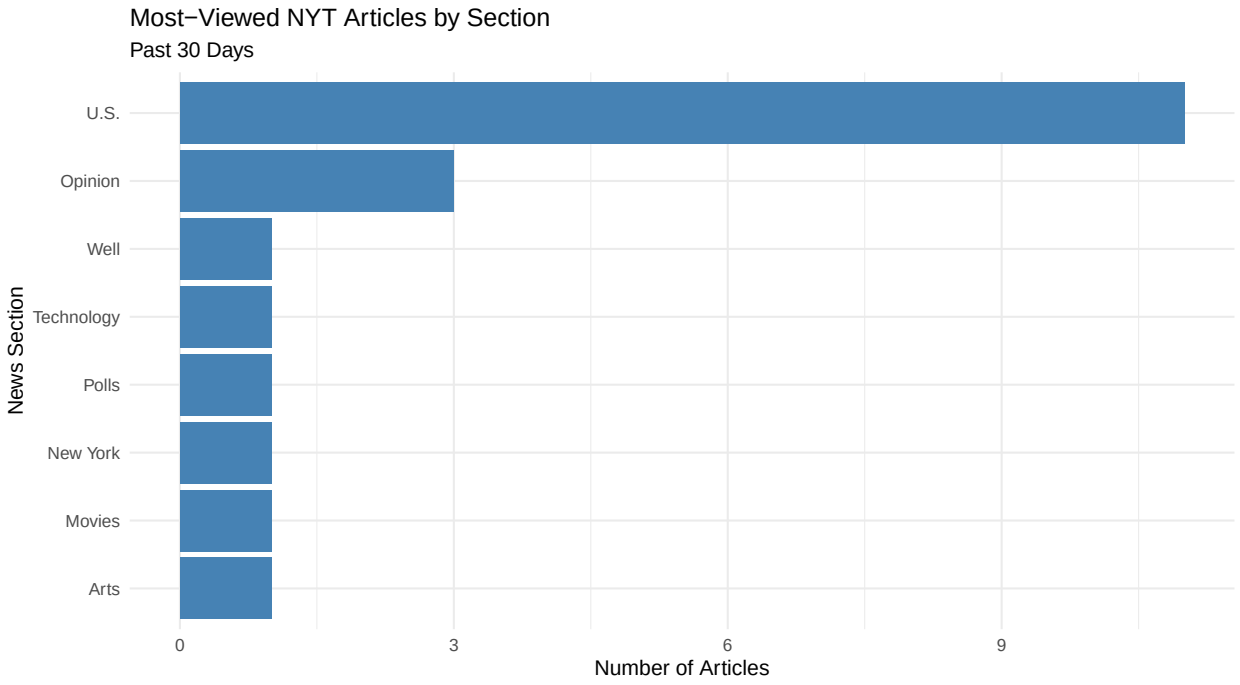
Table 4: Top Five Most-Represented NYT Sections

section	article_count	percentage
U.S.	11	55
Opinion	3	15
Arts	1	5
Movies	1	5
New York	1	5

Data Visualization

The following chart compares the number of most-viewed articles in each section.

```r
ggplot(
  section_summary,
  aes(
    x = reorder(section, article_count),
    y = article_count
  )
) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Most-Viewed NYT Articles by Section",
    subtitle = "Past 30 Days",
    x = "News Section",
    y = "Number of Articles"
  ) +
  theme_minimal()
```



Findings

The API returned **20 articles** across **8 news sections**.

The section with the highest representation was **U.S.**, containing **11 articles**, or **55%** of the returned articles.

The second most represented section was **Opinion**, with **3 articles**.

The five most represented sections in the current results were: **U.S., Opinion, Arts, Movies, New York**.

Based on the API results collected during testing, U.S. articles represented 55% of the 20 returned articles, followed by Opinion at 15%. The other six sections each represented 5%. These figures may change when the report is run again.

Conclusion

The analysis demonstrates how an API can be used to retrieve semi-structured JSON data and transform it into a clean data frame for analysis. The results indicate that the U.S. section had the highest representation among the most-viewed articles returned during testing.

However, the API only returns a selected list of popular articles, not all published articles or their actual page-view counts. Therefore, the findings describe section representation within the returned list rather than overall readership.

In the future, I could collect results at regular intervals and compare changes in the most frequently represented sections over time.

References

New York Times. (2026). *Most Popular API*. https://developer.nytimes.com/docs/most-popular-product/1/overview